Introduction to Algorithm

- COURSE DESCRIPTION
- **Why study algorithms?**
- Learn Computational Thinking first
- What study algorithms?
- How study algorithms?

Subject: Introduction to Algorithm

Lecturer: Hoang Do Thanh Tung

Credit points	**3 ECTS**
Level	Undergraduate
Teaching time **Location**	University of Science and Technology of Hanoi

COURSE DESCRIPTION

Course Goals

- Provide an introduction to the field of algorithms.
- Help students taking their first algorithm course understand
  - what algorithms are,
  - why we study them, and
  - how to approach algorithmic thinking.
- Develop the ability to express algorithms clearly using pseudo-code and visual diagrams.

Text Book and Reference Books

- Text Book:
  - Jeff Erickson. Algorithms. Independently published. 2019.
  - Mark A. Weiss . **Data Structures & Algorithm Analysis in C++** 4th Edition. Pearson, 2014.
- Reference Books:
  - Sanjoy Dasgupta. Algorithms. McGraw-Hill, 2006.
  - Cormen, Leiserson, Rivest, Introduction to Algorithms, 2nd Ed., MIT Press, 2001.

Examination and Grading

- Examination
  - A midterm exam with an assignment
  - A final exam
- Grading
  - Attendance/Attitude 10%
  - Class exercise and Lab practices 20%
  - A midterm exam 20%
  - Final exam 50%

Course Prerequisites

- No prior programming experience is assumed. However, students are expected to be familiar with the following concepts:
  - **Fundamental data types**, such as decimal numbers
  - **Basic programming concepts**, including variables, conditionals, loops, and recursion (conceptual understanding only)
  - **High school-level algebra**

Course plan

- Class Schedule (1 week of theory, 1 week of practice, alternating)
  - Theory: 3 hours per lesson, 1 lesson per week
  - Practical: 2 hours per lesson
- Material
  - moodle.usth.edu.vn
- Modeling Tool for practice
  - https://app.diagrams.net/
- Use Chatting for QA to chat and submit your practices
- Each student uses the paper with their computers or smartphones to do practice

Rules to study this course



- You can easily find my slides, course books, and other materials online.
  - So there's no need to rush through all the slides.
  - Our goal is to understand each concept as we go.
  - We learn better by exploring examples on our own.
- Don’t ask me directly for the solutions
  - try to analyze the problems first to gain your own understanding.
  - Try to find the solutions by yourself,
  - and then we’ll discuss them together.

Why study algorithms?



Real-world Applications of Algorithms

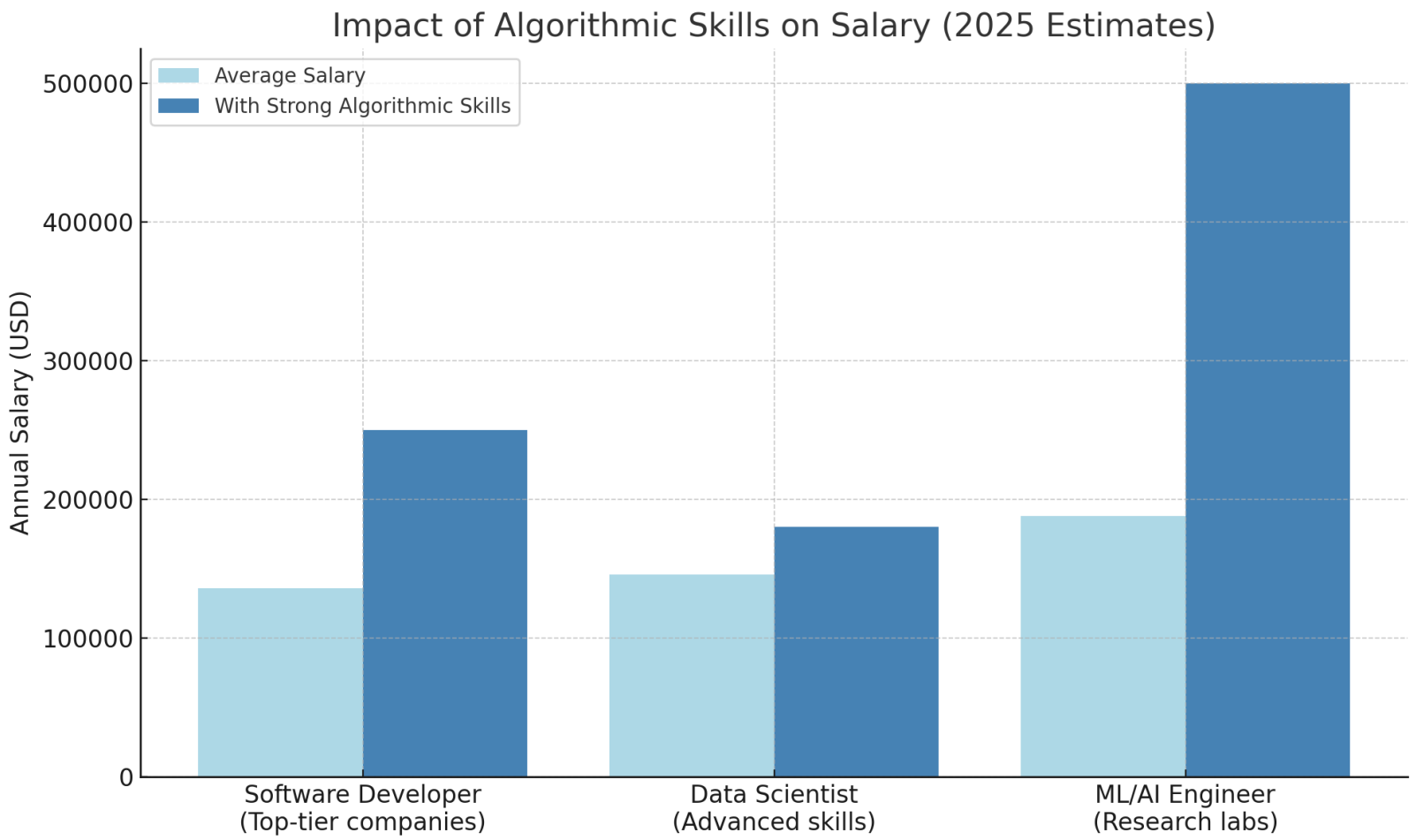
- **Search engines**: Google finds what you need in milliseconds — thanks to smart ranking algorithms.
- **Maps & GPS**: Apps like Google Maps use algorithms to find the fastest route based on traffic and road conditions.
- **Social media feeds**: Instagram, TikTok, and Facebook decide what you see next — using algorithms that learn your behavior.
- **Banks and fintech:** Apps like MoMo or online trading platforms like VNDirect, SSI Smart Trading – use algorithms to manage risk and recommend financial decisions..
- **Streaming platforms**: Netflix and Spotify recommend content based on what you've watched or listened to — all algorithm-driven.

Why Should We Learn Algorithms?

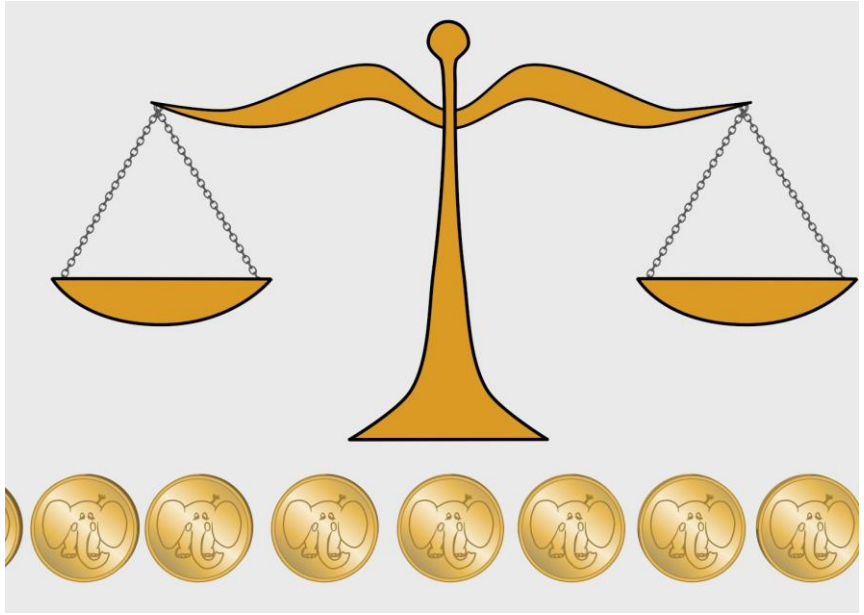
- 🔍 **Train your brain to solve problems** — like a detective, but smarter and faster.
- 🧠 **Think like a computer scientist** — structured, logical, and creative at the same time.
- 🚀 **Make better decisions** — in life, in code, and in business.
- 📊 **Understand data** — not just collect it. Learn to see what others miss.
- 🤖 **Automate the boring stuff** — let machines do the repetitive tasks for you.
- 💡 **Turn ideas into reality** — from smart apps to innovative tech solutions.
- 💼 **Open doors to exciting careers** — AI, fintech, cybersecurity, and more.

Salary Roles That Rely Heavily on Algorithmic Skills (2025 Data)

- **Software Developer (Algorithm-Oriented)**
  - **Average salary**: ~$136,000/year
  - **With strong algorithmic skills**: Up to **$180,000–$250,000/year** in top-tier companies (e.g., Google, Meta, DeepMind).
  - **Why algorithms matter**: Writing optimized code, designing scalable systems, choosing the right data structures and paradigms.
- **Data Scientist**
  - **Average salary**: ~$146,000/year
  - **With advanced algorithmic skills (e.g., in ML or graph data)**: $180,000+/year
  - **Why algorithms matter**: Data preprocessing, clustering, recommendation systems, and time-series forecasting all rely on algorithmic thinking.
- **Machine Learning / AI Engineer**
  - **Role focus**: Algorithm design for model training, optimization (e.g., gradient descent, reinforcement learning), and deployment.
  - **Average salary**: ~$188,000/year
  - **At top AI research labs (OpenAI, DeepMind)**: Up to $300,000–$500,000/year total comp.
  - **Why algorithms matter**: ML is built on algorithms — from learning rules to efficient inference.


Impact of Algorithmic Skills on Salary (2025 Estimates)
Average Salary
With Strong Algorithmic Skills
Annual Salary (USD)
500000
400000
300000
200000
100000
0
Software Developer
(Top-tier companies)
Data Scientist
(Advanced skills)
ML/AI Engineer
(Research labs)

Try some algorithms



- **Fake Coin Puzzle**
  - You have **8 identical-looking coins**.
    One of them is **fake** and is **slightly lighter** than the others.
  - You have a **balance scale** (it tells you only heavier, lighter, or equal — no numbers).
- How would you find the fake one? What's your plan?
- Thuật toán tìm kiếm: https://colab.research.google.com/drive/1q1OMP4QGGchapZHyu9rXAHCAaQlzYYkn?usp=sharing

Hint: Use **divide and conquer** to reduce the number of possibilities efficiently.

LEARN COMPUTATIONAL THINKING

the way we solve problems like a computer scientist

Types of Thinking

"Thinking" is how we use our minds to deal with problems, information, or ideas

Type of Thinking	Description	Example
Critical Thinking	Analyzing and evaluating ideas or arguments logically	Identifying flaws in a policy or news article
Creative Thinking	Generating new, innovative ideas	Brainstorming ideas for a startup or a game
Logical Thinking	Reasoning step-by-step with rules or principles	Solving a Sudoku puzzle or proving a math theorem
Computational Thinking	Structuring and automating solutions to complex problems in a way that a computer (or human) could execute	Breaking down a task into repeatable steps, like sorting emails or analyzing traffic data
Design Thinking	User-centered, iterative problem solving	Creating a mobile app interface based on user feedback
Strategic Thinking	Planning ahead with goals, risks, and resources in mind	Choosing a career path or managing a business strategy

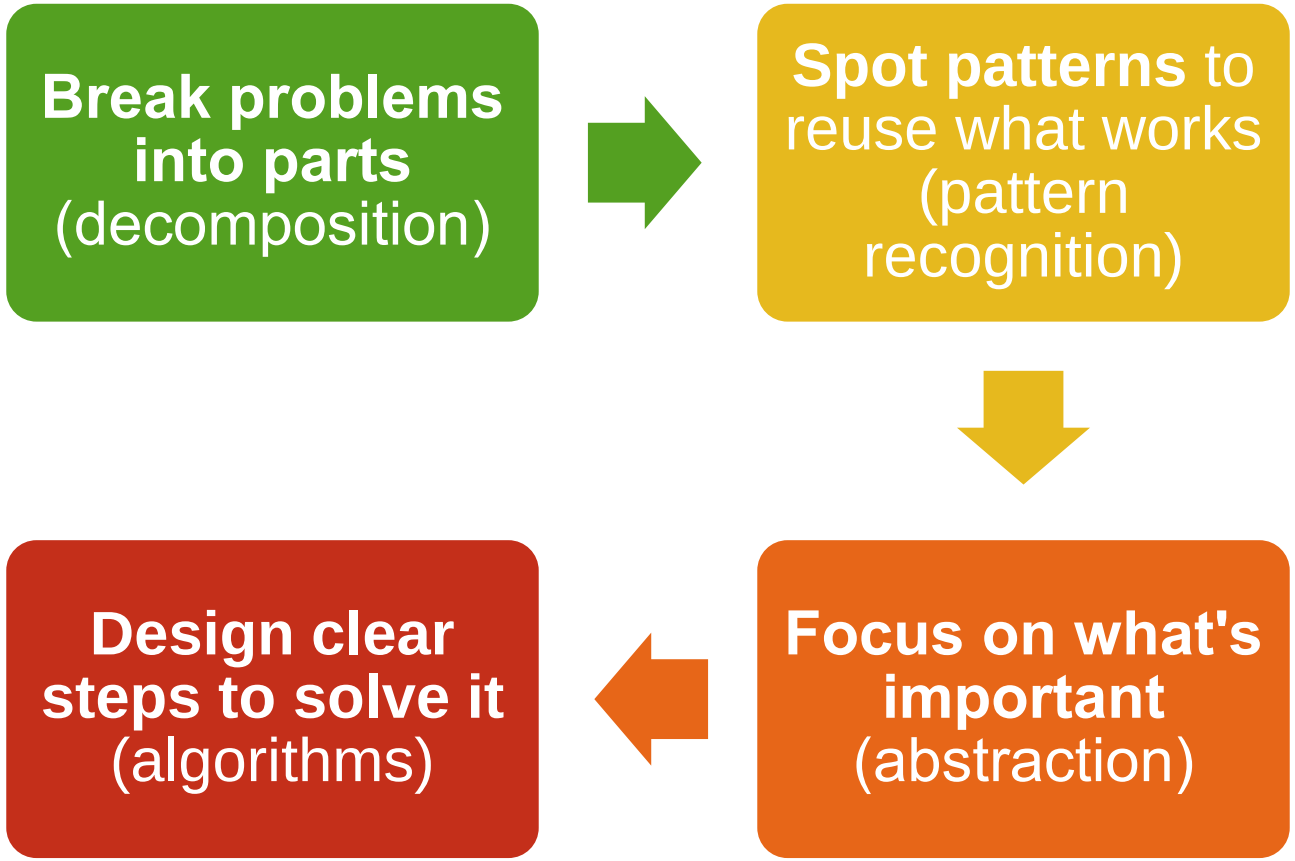
Why Learn Computational Thinking?

Computational thinking teaches you how to break complex problems into smaller, manageable steps.

It helps you **abstract**, **analyze**, and **automate** — whether you're writing code or designing real-world systems.

It's the foundation behind **algorithms**, **AI**, **data science**, and even **everyday decision-making**.

It teaches you how to solve problems systematically



It applies beyond computer science

In education: plan study schedules, analyze grades, automate practice routines

In business: automate reports, optimize workflows

In daily life: recipes, travel planning, even packing for a trip = algorithmic steps!

It builds future-ready skills

AI, robotics, big data, cybersecurity — all need CT

Problem-solving, automation, and logical reasoning are key in any 21st-century job

It makes you not just a tech user, but someone who understands **how the systems work**

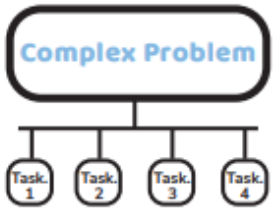
CT is a Foundational Skill in the Digital Age

- **In education & technology:**
  - CT is the **foundation of computer science**, software development, data science, and artificial intelligence.
  - Whether you're working with software, robotics, bioinformatics, or finance, **algorithmic thinking and problem decomposition** are essential.
- **In everyday life & modern jobs:**
  - Managers with CT skills **organize processes more efficiently**.
  - Marketers with CT **analyze customer behavior more effectively**.
  - CT helps people **think clearly, reduce complexity, and solve problems logically** — even without writing code.

CT is a Core Skill for Adaptation and Innovation

- **In the age of AI and automation:**
  - Repetitive jobs are being replaced by automation and AI.
  - The people who thrive will be those who can **analyze, abstract, and solve new problems** — in other words, **computational thinkers**.
  - You don't have to compete with AI — but you **do need to understand how it thinks**.
- **In global education and policy:**
  - CT is being taught at the **K-12 level** in countries like the US, Singapore, and South Korea.
  - Organizations like **UNESCO, OECD, and the World Economic Forum** promote CT as a **21st-century core competency**.
- **In innovation and cross-disciplinary work:**
  - CT bridges fields: **AI + medicine**, **CT + biology**, **CT + design**, and more.
  - The most impactful innovations will come from those who can apply computational thinking to **non-technical domains**.

Key methods in Computational thinking



- Computational thinking involves taking that complex problem and breaking it down into a series of small, more manageable problems (**Decomposition**).



- Each of these smaller problems can then be looked at individually, considering how similar problems have been solved previously (**Pattern Recognition**).

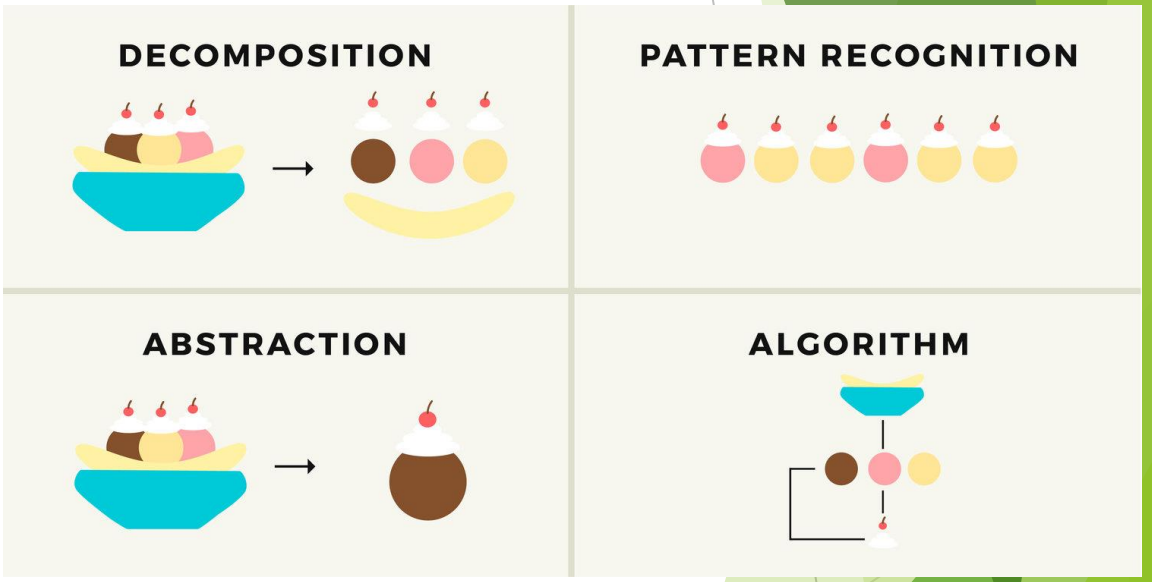


- And focusing only on the important details, while ignoring irrelevant information (**Abstraction**).





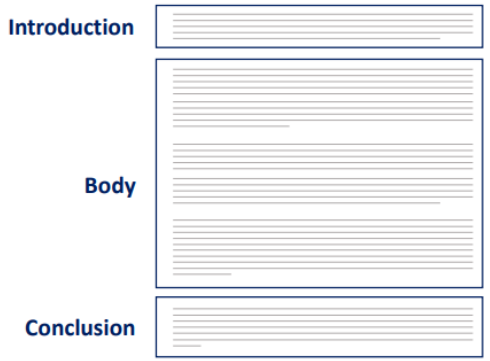
- Next, simple steps or rules to solve each of the smaller problems can be designed (**Algorithms**).

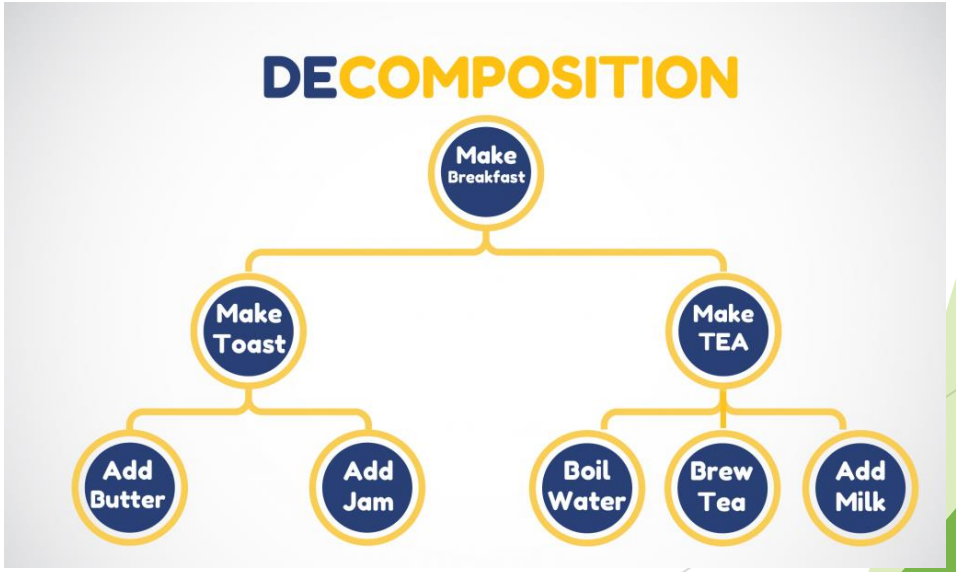


Decomposition

- Tt is breaking down large complex problems into smaller problems that are easy to understand and to manage
- **Benefits:** Makes problems less overwhelming
  - Helps identify the core issues
  - Facilitates step-by-step solutions

Decomposition: Outlining a Paper





Example for Decomposition

- **For example**: you have been late to school 3 times this week. You'd like to fix the problem and be on time starting tomorrow. To start with, define the problem.
- **Problem definition**: Late to school multiple times.
- **Solution we would like**: Change morning process so you can't be late for school ever again!
- **Decomposition**:
  - Why were you late for school? I missed the school bus ;
  - Why did you miss the bus? It leaves exactly at 7:26 AM and I reached there a few minutes late for it ;
  - Why were you late by a few minutes? It takes me too long to get all my stuff together (backpack, pack lunch bag, fill water bottle, put on shoes, etc.) in the morning.



More examples

- Planning a birthday party
- Organizing a class trip
- Cooking a meal
- Joining a competition

Bigger Problems

- Organizing a School or University Event
- Building a Simple Website or App
- Preparing for University Applications
- Building a Search Engine (like Google)
- Training an AI to Play a Game (e.g., Chess or Go)
- Secure a Communication System (End-to-End Encryption)
- Building an E-Commerce Website (e.g., Shopee, Amazon)

Do exercise

- Problem: Plan a summer vacation trip for your family in next month
- Break it down into small smaller, more manageable parts

Break down

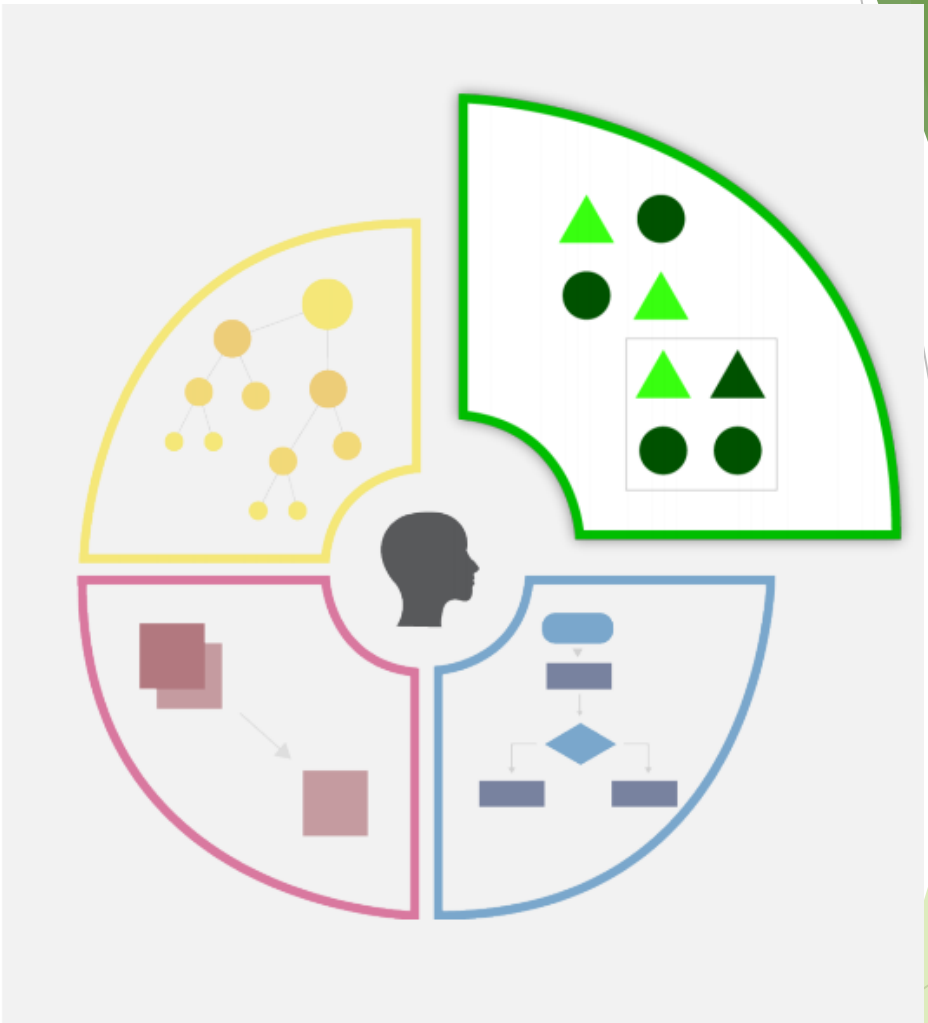
Sub-Task	What do we need to plan?	Notes / Why it's important
1	Choose destination	Beaches? Mountains? Abroad?
2	Set the budget	Total cost per person?
3	Decide trip dates	When is everyone free?
4	Book transportation	Train, plane, carpool?
5	Book accommodation	Hotel, hostel, homestay?
6	Plan daily activities	Sightseeing, food, games, etc.
7	Handle packing and checklist	What to bring? Shared items?

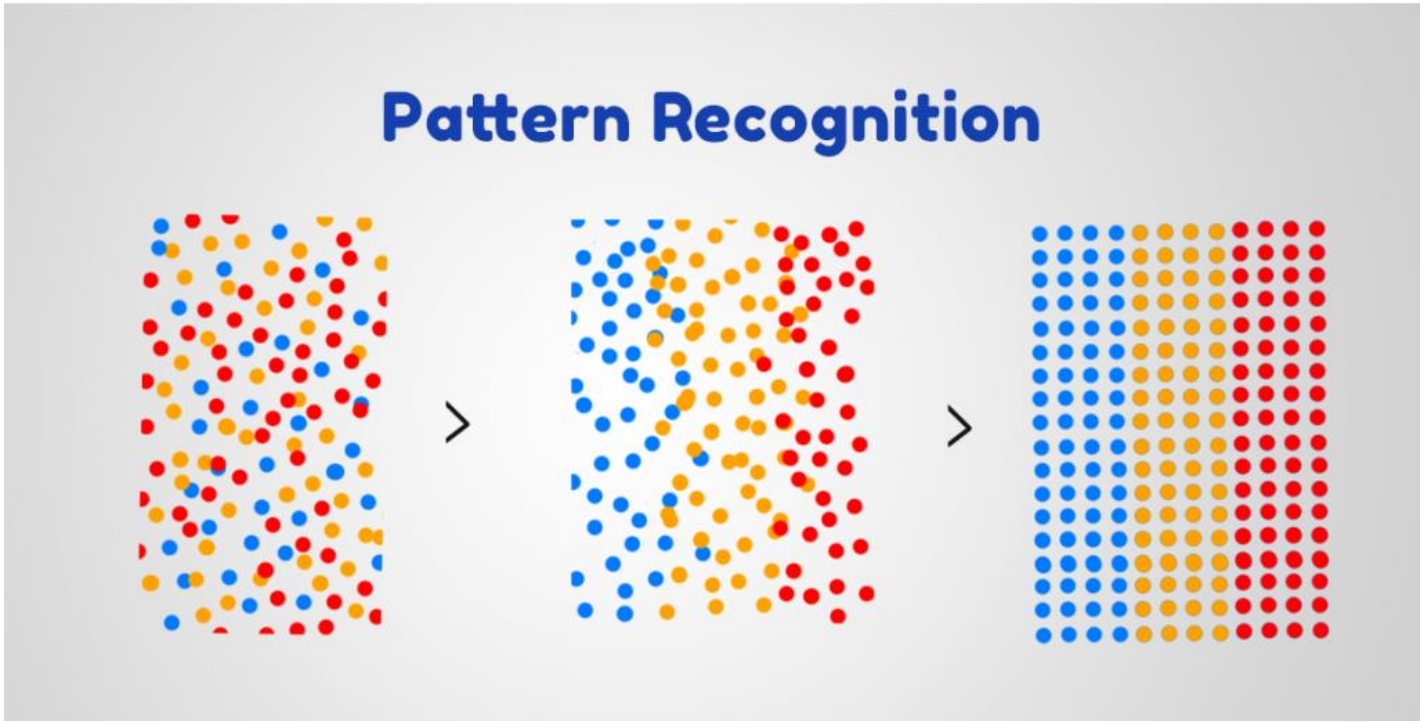
Propose Solution Ideas

Sub-Task	Solution Ideas or Tools
1	Google search + vote with friends
2	Use Excel or Splitwise
3	Google Calendar or Doodle poll
4	Skyscanner, Traveloka, or Grab
5	Airbnb or Booking.com
6	Make a shared itinerary in Google Docs
7	Checklist app (e.g., Notion, Trello)

Pattern Recognition

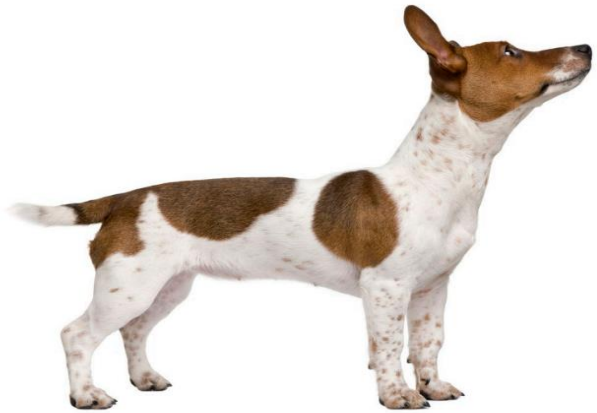
- Finding similarities or shared characteristics within or between problems
- Makes the problem easier to solve since the same solution can be used for each occurrence of the pattern





Pattern Recognition

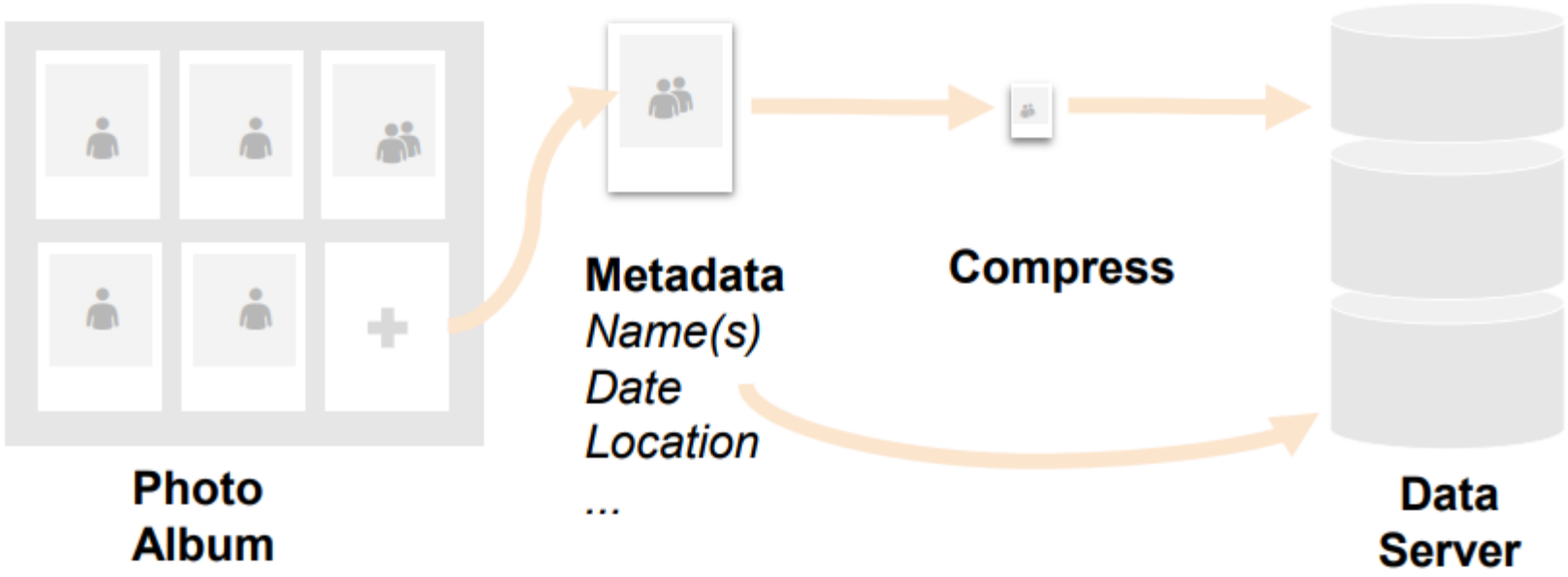
Drawing Dogs



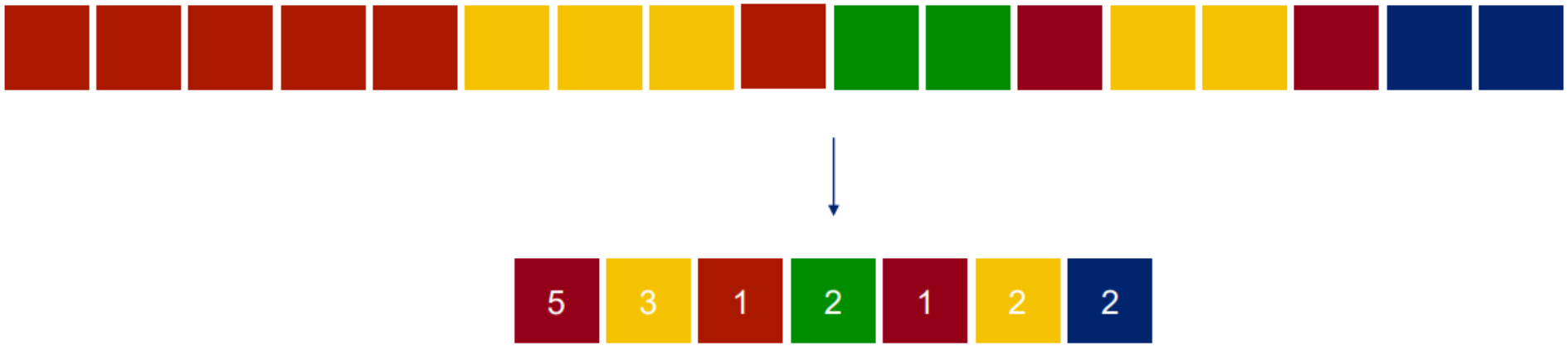
Drawing Different Dogs



Social Media Site: Photo Albums



Data Compression



Do Exercise

- Problem: The following sequence is a pattern of numbers. Can you identify the pattern and predict the next number in the sequence?

1, 2, 4, 7, 11, 16, ...

Solution:

- By examining the sequence, we notice that the difference between adjacent terms is increasing by 1, starting from 1.

1, 2-1, 4-2, 7-3, 11-4, 16-5, ...

- The pattern of adjustment:

0, 1, 2, 3, 4, 5, ...

- So the next number in the sequence would be 16 + 6 = 22.
- By practicing pattern recognition exercises, you can recognize patterns and relationships in data, which is a key component of computational thinking.

Data Representation & Abstraction

- Determining what characteristics of the problem are important and filtering out those that are not
- Use these to create a representation of what we are trying to solve



Students in a University Setting



Data Representation: Students

Important:

- name and billing address
- student id
- on-campus address
- phone number
- …

Not Important:

- favorite color
- shoe size
- food preferences
- …

Data Representation: Books

Important:

- author list
- title
- ISBN
- publication date
- edition
- category
- ratings
- summary
- ...

Not Important:

- color of the cover
- birthplace of authors
- complete contents of the book
- ...



Do Exercise

- **Problem:** Suppose you are planning a picnic with your friends and want to make sure you have everything you need. You decide to create a list of all the items you will need for the picnic. Using data abstraction, create a list of items for the picnic.

Solution:

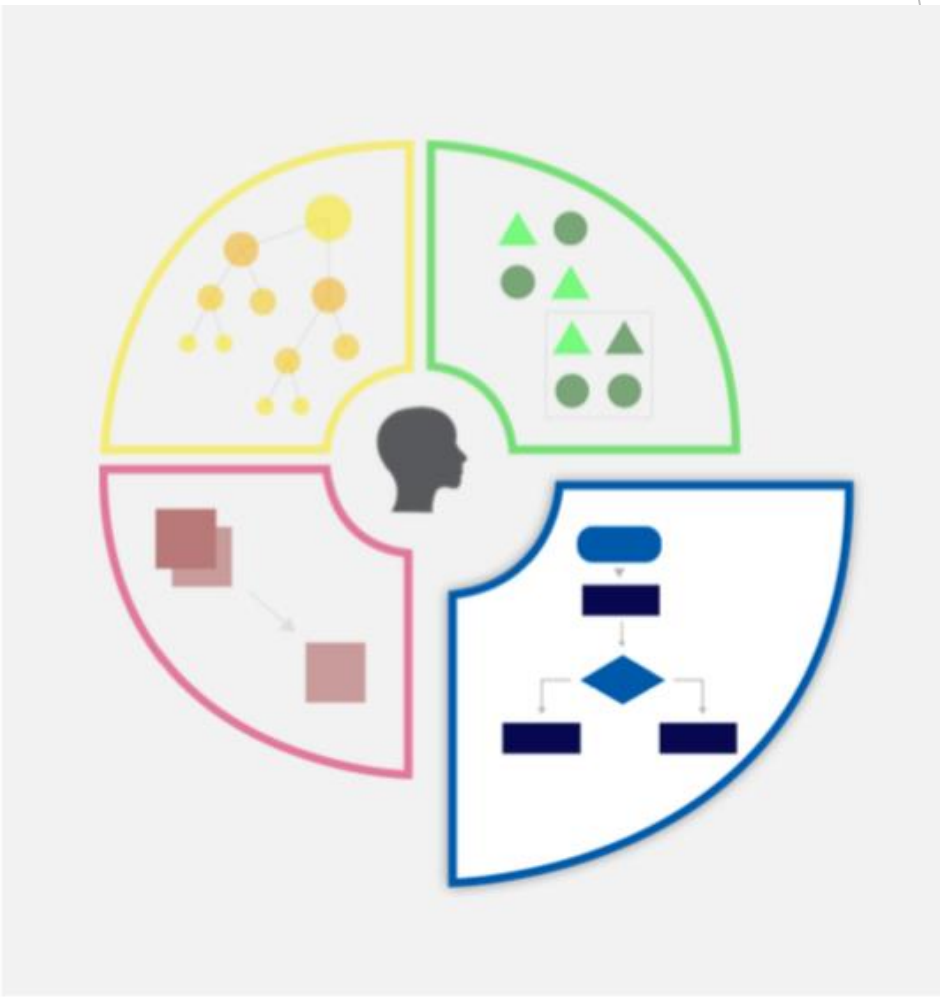
Category: Food and Drink
- Sandwiches
- Chips
- Fruit
- Vegetables
- Water
- Soft drinks

Category: Supplies
- Blanket
- Plates
- Cups
- Utensils
- Napkins

Category: Recreation
- Frisbee
- Soccer ball
- Deck of cards

Algorithms

- Step-by-step instructions of how to solve a problem
- Identifies what is to be done (the instructions), and the order in which they should be done.



Create an algorithm to make a cup of tea

Making a Cup of Tea

- Fill electric tea kettle
- Bring it to a boil
- Pour hot water in cup
- Put teabag in cup
- Steep for 4 minutes
- Remove teabag



Do Exercise

- Problem: Create an algorithm for washing dishes.

Solution:

1. **Setup:**
    1. Gather all dirty dishes by the sink.
    2. Fill the sink with warm water and detergent.
2. **Washing:**
    1. Scrub each dish in the sink,
    2. starting with the cleanest and moving to the dirtiest.
    3. Rinse off the soap under running water.
3. **Drying:**
    1. Dry each dish with a clean towel and
    2. place it in the dish rack.
4. **Cleanup:**
    1. Once all dishes are done, drain and rinse the sink.
    2. Wipe down the sink and counter.
    3. Store away dishes and cleaning supplies.

Benefits of Learning Thinking Skills Before Algorithms

- **Better Understanding of Problem Solving:**
  - Skills like **decomposition** and **abstraction** help them break down a problem and focus on what really matters
  - If you don’t know what the input/output should look like, no algorithm will help
- **Pattern Recognition Builds Efficiency:**
  - **Reuse logic** instead of starting from scratch every time
  - Students who recognize that “finding the maximum value” is a common pattern will more easily understand selection or bubble sort
- **Abstraction Improves Focus:**
  - **filter out unimportant information**, which is a core part of writing efficient and elegant code.
  - When designing a game, you don’t need to simulate gravity perfectly — just enough to make movement feel real.
- **Foundation for Building Algorithms**
  - Without thinking skills, students may try to **memorize algorithms blindly**
  - Instead of "copy-paste code," students can **design their own algorithms**

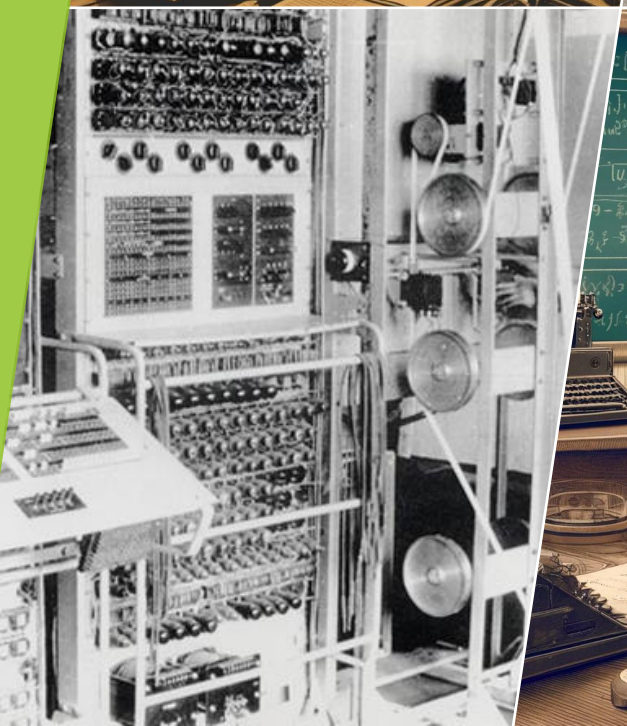
What are algorithms?

An **algorithm** is a **step-by-step set of instructions** to solve a problem or complete a task

What do you want in your life, to be fed or fish?









Algorithm: A brief History

- Muhammad ibn Musa al-Khwarizmi, one of the most influential mathematicians in 9th century in Baghdad, wrote a textbook in Arabic about adding, multiplying, dividing numbers, and extracting square roots and computing π.
- Many centuries later, decimal system was adopted in Europe, and the *procedures* in Al Khwarizmi's book were named after him as "Algorithms".
- Alan Turing formalized Turing Machine in 1936, the foundation of *computer science*.

What is an Algorithm?

- An algorithm is a sequence of instructions solve a problem
  - Explicit
  - Precise
  - Unambiguous
  - Mechanically-executable
- A good algorithm
  - it solves a problem in a reasonable amount of time and
  - produces the correct result for all inputs.
- Algorithms can be expressed in many different ways, pseudocode, flowcharts, and programming languages

A good algorithm

- A good algorithm is like a perfect recipe: all the steps are there (**explicit**), each one is clear (**precise**), there's only one way to read it (**unambiguous**), and even a robot could follow it (**mechanically-executable**)

Term	Meaning	Focuses on...	What goes wrong if missing?
Explicit	All steps are clearly written — nothing is missing	Having every step included	You skip steps or forget what to do
Precise	Each step is detailed with exact values	The details of each step	Instructions are too vague
Unambiguous	Each step has only one meaning	No confusion in interpretation	Different people/machines understand it differently
Mechanically-executable	A machine (or person) can follow the steps exactly	Steps are automatic, logical	Needs human judgment — not usable by a computer

Example: Using an ATM to Withdraw Cash

- Algorithm:
  - Insert your card into the ATM.
  - Enter your 4-digit PIN.
  - Choose “Withdraw Money”.
  - Select the amount: 100, 200, 500, or 1,000.
  - Confirm the transaction.
  - Take the cash.
  - Take your card.
- This is a good algorithm because it is:
  - **Explicit** – Every step is included
  - **Precise** – No vague instructions
  - **Unambiguous** – Each action has one meaning
  - **Mechanically-executable** – A machine (ATM) can follow this

- Algorithm:
  - Insert your card into the ATM.
  - Enter your Code
  - ~~Choose “Withdraw Money”.~~
  - Pick the amount you feel is appropriate
  - Confirm the transaction.
  - Take the cash.
  - Take your card.
- This is a bad algorithm because it is Not:
  - **Explicit** – No steps on how
  - **Precise** – What code? PIN? Account number?
  - **Unambiguous** – How? Can I type it? Select it?
  - **Mechanically-executable** – A machine can't “feel”

Exercise: Finding the maximum value in an array of numbers

- Algorithm

Initialization:

- The `max` variable is initialized to the value of the first element in the array.

Loop:

- The loop continues until all the elements in the array have been checked.
- In each iteration:
  - Compare the current element of the array with the current value of `max`.
  - If the current element is greater than `max`, update the value of `max` to the current element.

Termination:

- After the loop ends, the value of `max` will be the maximum value in the array.

Problem, Algorithm, Mathematics?



You have to pick your sisters up at 5pm everyday with $4 budget

Bus ($2)
Taxi, Grab bike (> $3)
Speed >= Subway





Subway: 30 minutes, 4.30PM (last)
Price: $2



Time to leave varies: 4.15-4.45pm

Problem Statement

Your mother has requested that you pick up your sisters from school every day at 5:00 PM, with a budget of $4. The constraints include:

Your departure time from university can vary between 4:15 PM and 4:45 PM.

Transportation options include:

- Bus: $2, but slow
- Taxi or Grab: More than $3, faster
- Subway: $2, takes 30 minutes, last one leaves at 4:30 PM

Objective

- Develop an algorithm that ensures you can always pick up your sisters by 5:00 PM, considering the varying departure times and transportation costs and speeds.

When Do We Need Mathematics?

- **Calculating Travel Times**: Understanding when each transportation method will get you to your destination.
- **Cost Analysis**: Keeping within a $4 budget while considering various options.
- **Decision Making**: Comparing costs and benefits to make the best choice based on quantitative data.

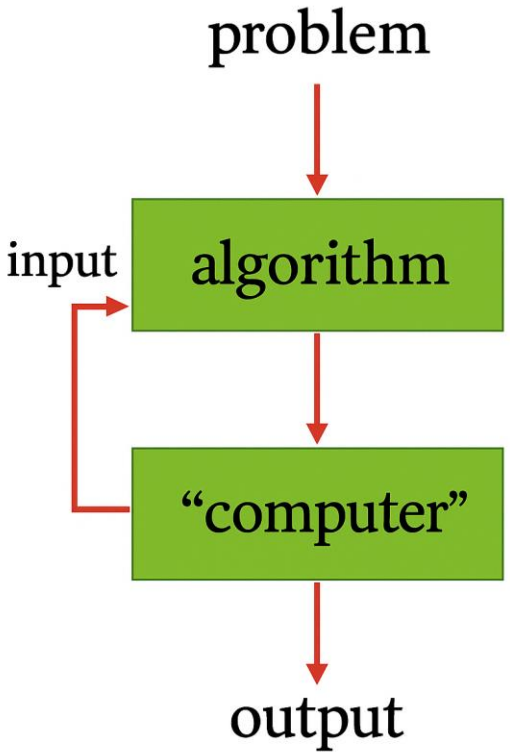
Algorithm

1. **Input**: Time t you leave the university.
2. **Conditions**:
    1. If t ≤ 4:30 PM, you can choose between the subway or other modes based on cost and time.
    2. If t > 4:30 PM, subway is not an option.
3. **Decision Criteria**:
    1. Always prefer the cheapest option that gets you there by 5:00 PM.
    2. If leaving after 4:30 PM, the only options are bus or taxi/grab.
4. **Process**:
    1. If t ≤ 4:30 PM:
        1. Check if taking the subway allows you to arrive on time (i.e., 5:00 PM).
        2. If yes, choose subway if it is the cheapest option that meets the time constraint.
        3. If no or if subway is infeasible due to other constraints (e.g., budget), compare bus and taxi/grab for cost and feasibility.
    2. If t > 4:30 PM:
        1. Directly compare bus and taxi/grab based on which can meet the time requirement within budget.
5. **Output**: Best transportation mode based on the given time and constraints.

Is it an algorithm?

- Singing that a song ‘99 Bottles of Beer on the Wall’, for arbitrary values of 99:

```
BOTTLESOFBEER(n):
  For i ← n down to 1
      Sing "i bottles of beer on the wall, i bottles of beer,"
      Sing "Take one down, pass it around, i − 1 bottles of beer on the wall."
  Sing "No bottles of beer on the wall, no bottles of beer,"
  Sing "Go to the store, buy some more, n bottles of beer on the wall."
```



So What is an algorithm for Computer?

A sequence of instructions that tells the computer how to solve a particular problem

Properties of computer Algorithms

- Finiteness: must eventually terminate
- Definiteness: each step is *precisely* defined, there is *no* ambiguity
- Input: information which is required in order to start the algorithm
- Output: result of the algorithm, with some relation to the input
- Effectiveness: should be able to be emulated manually (paper and pencil)

An Example: calculating the factorial of a positive integer

1.Input a positive integer "n".
2.Set a variable "result" to 1.
3.For each integer "i" from 1 to "n", do the following: a.
Multiply "result" by "i".
4.Output the value of "result" as the factorial of "n".

- Finite: clear starting point and will eventually terminate after a finite number of steps.
- Definite: each step is precisely defined, leaving no room for ambiguity
- Input: a positive integer "n"
- Output: factorial of "n".
- Effective:
  - it can be performed manually using paper and pencil
  - easily implemented in a computer program to automate the calculation of factorials

Example of computer algorithms



- *Beginner-friendly* (Search, Sort)
  - 1. **Searching Algorithms**
    - Examples: Linear Search, Binary Search
    - Purpose: To find a specific item in a collection (e.g., a name in a list, a value in an array).
      - Searching is foundational, easy to visualize, and required by almost all algorithmic domains.
  - 2. **Sorting Algorithms**
    - Examples: Bubble Sort, Merge Sort, QuickSort
    - Purpose: To rearrange data into a specific order (ascending, descending, custom).
      - Sorting underpins many other tasks (binary search, optimization), and is a gateway to complexity analysis.

Example of computer algorithms

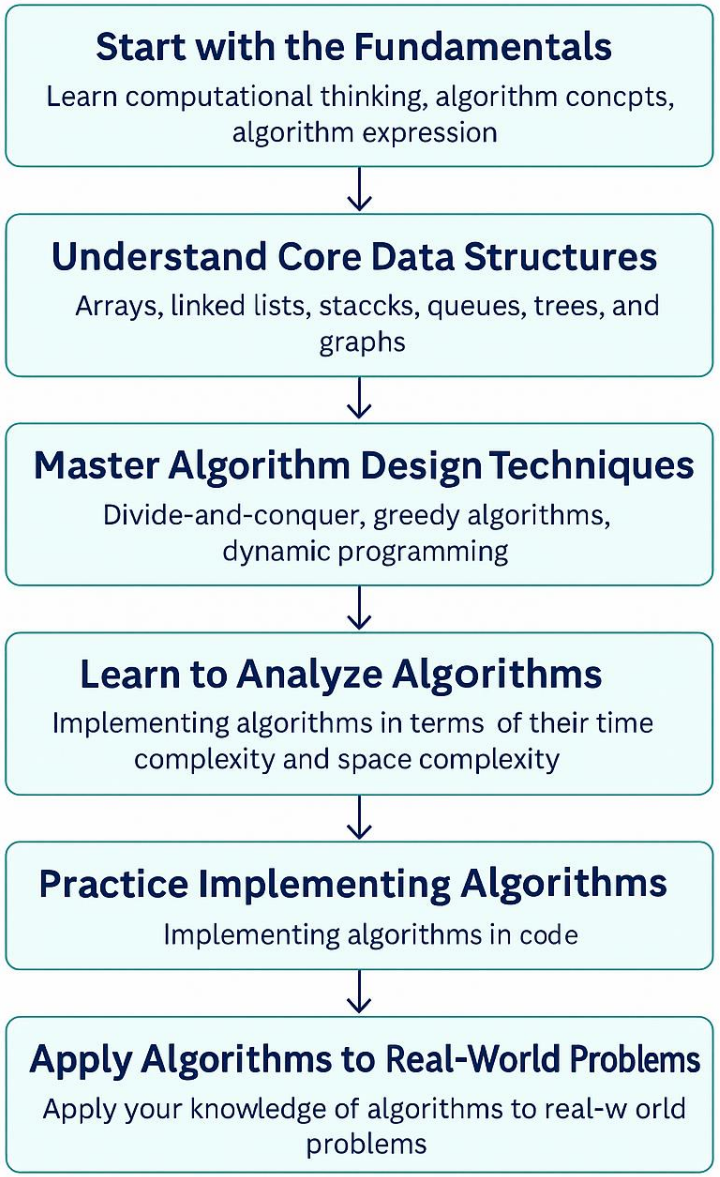
- 🔵 *Intermediate* (String, Graph)
  - 3. **String Algorithms**
    - Examples: Knuth-Morris-Pratt (KMP), Rabin-Karp
    - Purpose: To process and match patterns in text — finding substrings, comparing strings, detecting repetitions.
      - Slightly more abstract, but very common in real applications like search engines, plagiarism detection, etc.
  - 4. **Graph Algorithms**
    - Examples: BFS, DFS, Dijkstra’s, Bellman-Ford
    - Purpose: To solve network-related problems — shortest paths, connectivity, traversal.
      - Graphs represent relationships and systems — important in routing, social networks, AI.

Example of computer algorithms

- 🔴 *Advanced/Heavy computation* (Combinatorics, Geometry, Numerical)
  - 5. **Combinatorial Algorithms**
    - Examples: Traveling Salesman Problem (TSP), N-Queens
    - Purpose: To explore all (or best) combinations, permutations, or configurations.
      - These problems are NP-hard or exponential, require deeper logic (recursion, backtracking).
  - 6. **Geometric Algorithms**
    - Examples: Convex Hull, Line Intersection, Polygon Area
    - Purpose: To solve problems involving spatial relationships (e.g., maps, graphics, simulations).
      - Often requires combining math with algorithmic thinking (vectors, coordinates, cases).
  - 7. **Numerical Algorithms**
    - Examples: Matrix operations, numerical integration, root-finding
    - Purpose: To solve math-heavy problems in continuous domains — modeling, simulations, optimization.
      - Often depends on calculus or linear algebra background; used in engineering, ML, physics.

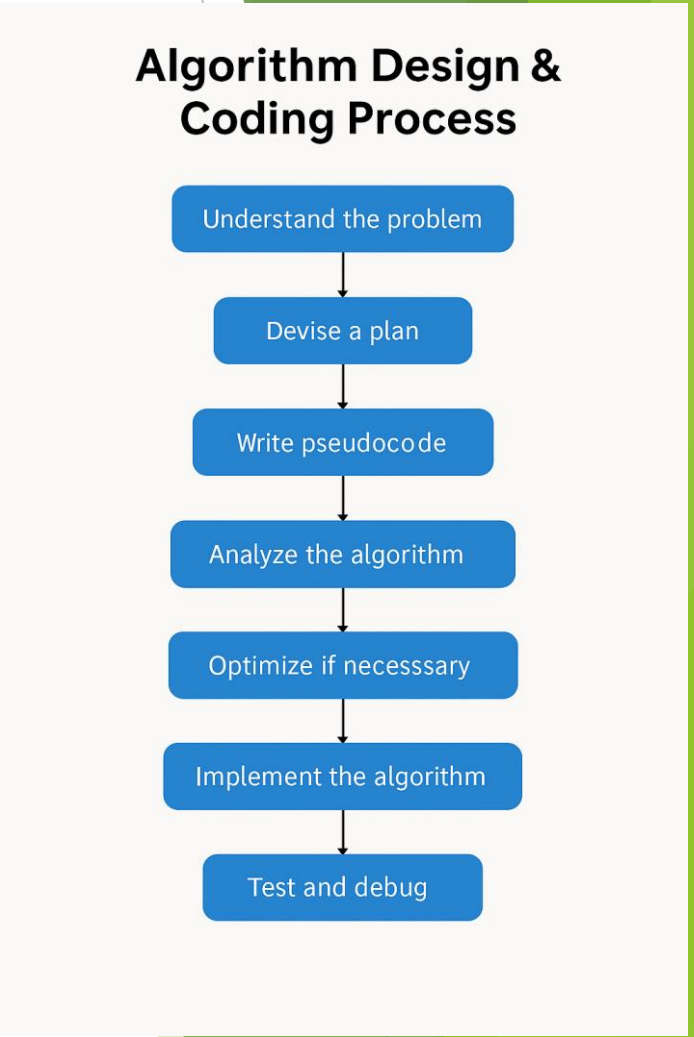
How to study algorithms?

Roadmap to Studying Algorithms



Algorithm Design and coding Process

- **Problem Solving**
  1. **Understand the Problem:** Identify inputs, outputs, constraints, and goals.
  2. **Devise a Plan:** Choose a strategy (e.g., brute force, divide-and-conquer, greedy, etc.) to solve the problem.
  3. **Write Pseudocode or Diagrams:** Express your logic in a structured
  4. **Analyze the Algorithm:** Evaluate time and space complexity
  5. **Optimize if Necessary:** Improve your plan if it's too slow or uses too much memory.
- **Programming**
  1. **Implement the Algorithm:** Translate the pseudocode into actual code in your programming language.
  2. **Test and Debug:** Use various test cases to check correctness and fix errors.



Programming vs. Problem Solving

🧠 **Problem Solving**

- Understanding the problem
- Designing a logical, efficient solution
- The result: an **algorithm**

💻 **Programming**

- Translating the algorithm into a **code implementation**
- Using modern languages (Python, JavaScript, C++, etc.)
- Making the solution **run on a computer**

What are we studying in this course?

- Learn the Basics of Algorithms
  - Computational Thinking
  - Algorithm Concepts: Understand the definition of algorithms, their importance, and where they are used. Learn about the characteristics of correctness, efficiency, and finiteness.
  - Algorithm Expression: Master expressing algorithms through pseudocode, flowcharts, and clear, concise natural language.
- Learn Algorithm Analysis
  - Time Complexity Analysis: execution time of an algorithm increases with the input size, and learn to express this relationship using Big O, Big Ω, and Big Θ notations.
  - Best, Worst, and Average Cases: best, worst, and average cases—to evaluate performance comprehensively.
  - ~~Space Complexity Analysis: Understand how much memory space an algorithm uses in operation and how this scales with the size of the input.~~

Some Exercises

Is this a good algorithm? Why?

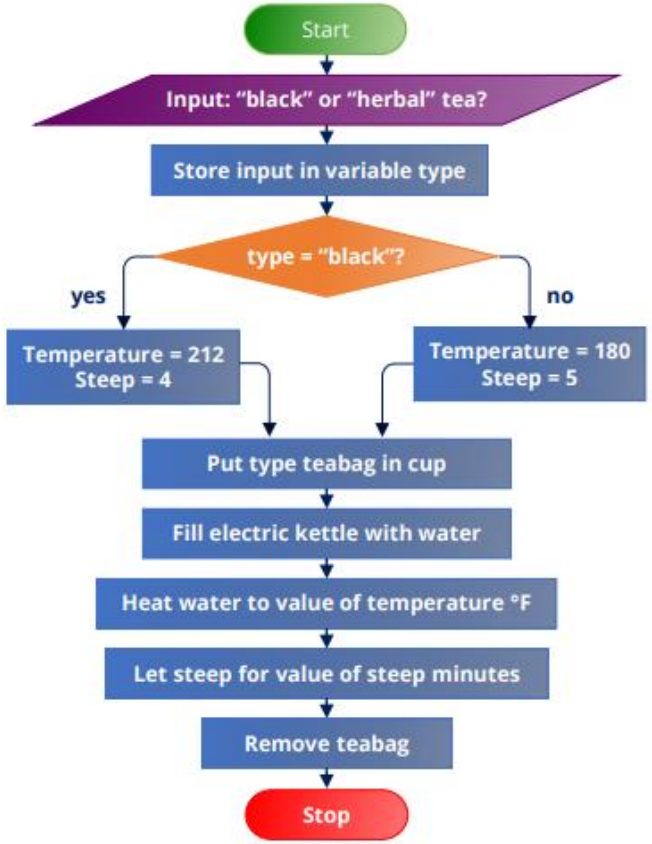
Making a Cup of Tea

- Fill electric tea kettle
- Bring it to a boil
- Pour hot water in cup
- Put teabag in cup
- Steep for 4 minutes
- Remove teabag

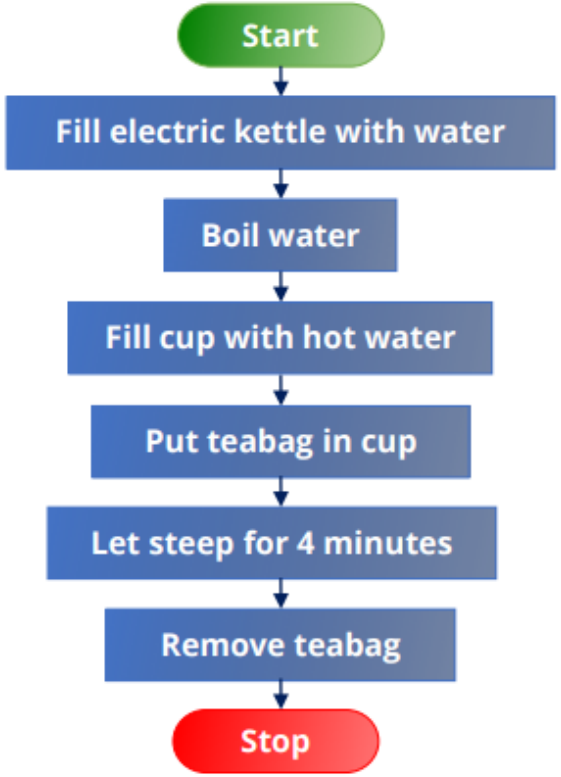


Find differences

Flowchart: Making a Cup of Herbal or Black Tea



Flowchart: Making a Cup of Tea



Is it a good computer algorithm? Why?

- Is the following a legitimate algorithm? (Finiteness, Definiteness, Input, Output, Effectiveness)

i ←1
While (i <= 10) do
a ← i + 1
Print the value of a
End of loop
Stop

Exercise: Compare Three GCD Algorithms

The greatest common divisor of two non-negative, not-both-zero integers m and n, denoted **gcd(m, n)**

Algorithm 1: Euclid

Steps:

1. Divide the larger number by the smaller.
2. Take the remainder.
3. Replace the larger number with the smaller number, and the smaller with the remainder.
4. Repeat until remainder = 0.
5. The last non-zero number is the GCD.

Example: Find GCD(60, 48)

- 60 ÷ 48 = 1, remainder **12**
- 48 ÷ 12 = 4, remainder **0**
  ✅ **GCD = 12**

Algorithm 2: Consecutive Integer Checking

Steps:

1. Start from the smaller of the two numbers.
2. Check if it divides both numbers.
3. If not, try the next smaller number.
4. Repeat until a number divides both.
5. That number is the GCD.

Example: Find GCD(60, 48)

- Try 48 → not divisible by 60
- Try 47… down to
- Try 12 → 60 ÷ 12 = 5, 48 ÷ 12 = 4
  ✅ **GCD = 12**

Algorithm 3: Middle School Procedure (Prime Factorization)

Steps:

1. Find the prime factors of each number.
2. List the factors.
3. Find the common prime factors.
4. Multiply them to get the GCD.

Example: Find GCD(60, 48)

- 60 = 2 × 2 × 3 × 5
- 48 = 2 × 2 × 2 × 3
- Common: 2 × 2 × 3 = **12**
  ✅ **GCD = 12**



What can we learn from the previous 3 examples?

- The same algorithm can be represented in several different ways. (different pseudo-codes)
- There might exists more than one algorithm for a certain problem.
- Algorithms for the same problem can be based on very different ideas and can solve the problem with dramatically different speeds.

Any question?